University of BRISTOL

Example Sizing Tool

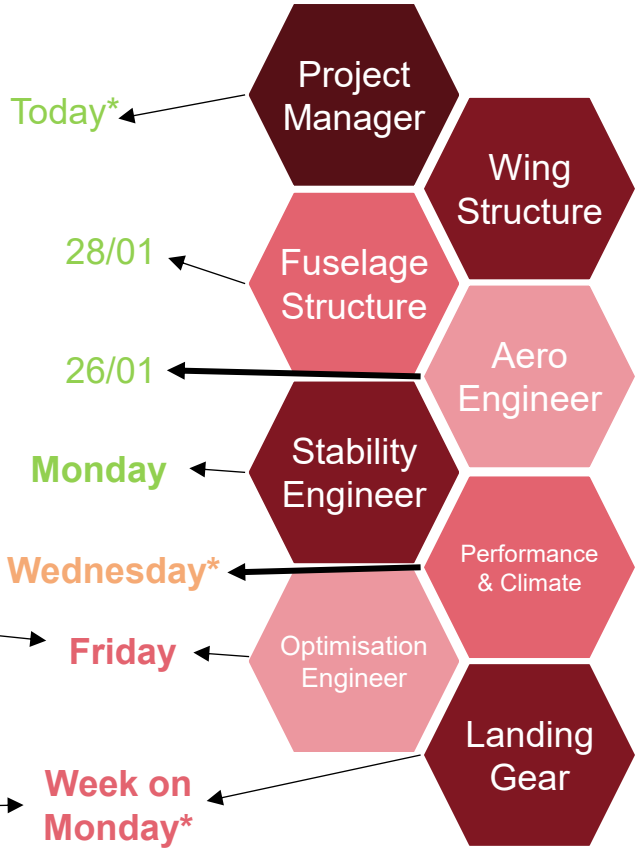
Evaluating and analysing the chosen concepts

CADEM0016

Unit Director: Fintan Healy (QB1.31) fintan.healy@bristol.ac.uk

bristol.ac.uk

Discipline Lectures Update



Constraint Analysis + Mission Analysis already covered

- "*Explore Design Space*" – Monte Carlo optimisation,
- Parameter studies (what are the most sensitive parameters)
- Engineering Design Optimization – too* comprehensive book

- Josh will upload a "*Getting Started Slide Deck*" today

Recap

Lecture 1

Turning customer needs into engineering requirements

What the aircraft must do

Lecture 2

Identifying concepts that can satisfy those requirements

Exploring possible ways to solve the problem

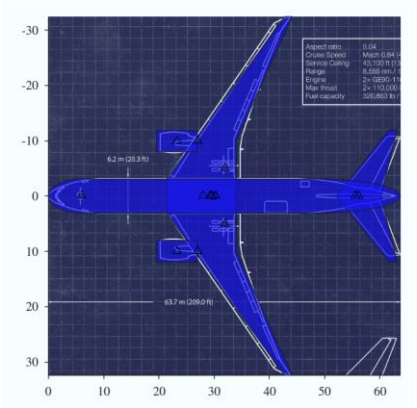
Today

Evaluating and analysing the chosen concepts

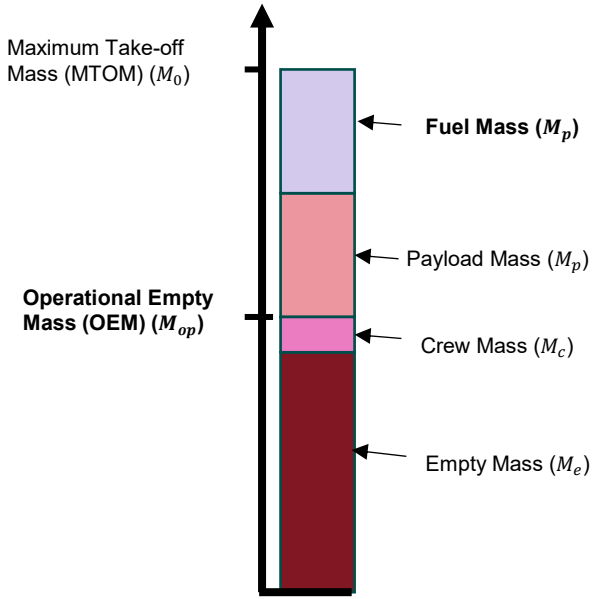
Understanding how well it works and where it falls short.

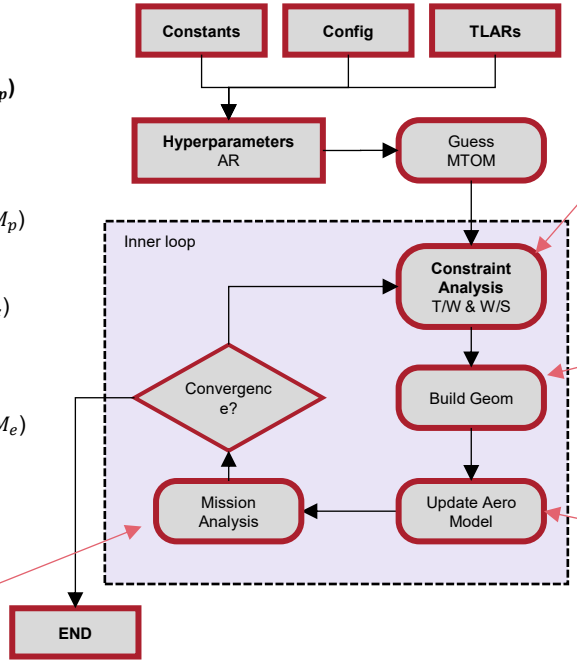
		Mass	Drag	Lift Capability	Take-off Distance	TRL	
Customer Requirement	**Weighting**	↘	↘	↗	↘	↘	Favourability
operating cost	0.4	2	9	9	0	5	
enviromental impact	0.42	3	8	1	0	0	
reliability	0.18	1	0	0	2	8	Sum
	Score	2.24	6.96	4.02	0.36	3.44	17.02
	Weighting	0.13	0.41	0.24	0.02	0.20	1

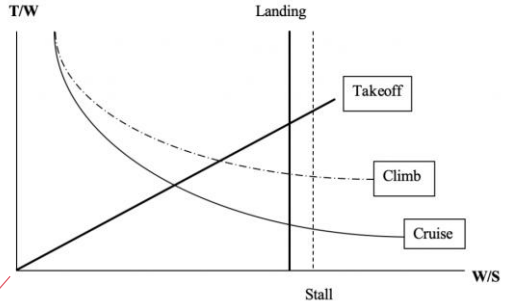
		Concept		
Criteria	**Weighting**	**A**	**B**	**C**
Mass	0.13	0	-2	0
Drag	0.41	0	3	1
Lift Capability	0.24	0	0	0
Take-off Distance	0.02	0	-1	0
TRL	0.20	0	-3	0
	Total	0.00	0.34	0.41
	Rank	3	2	1

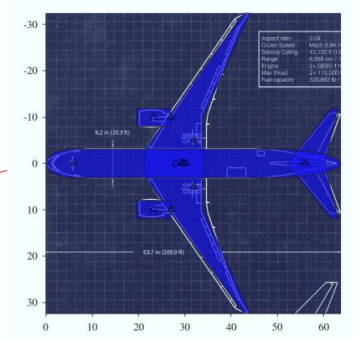


A Simple Sizing Flowchart









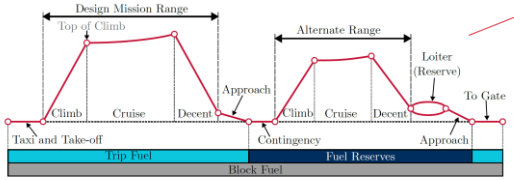
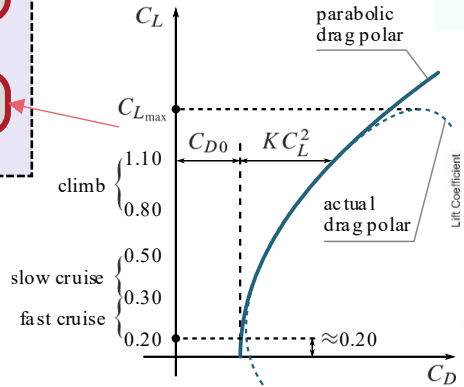
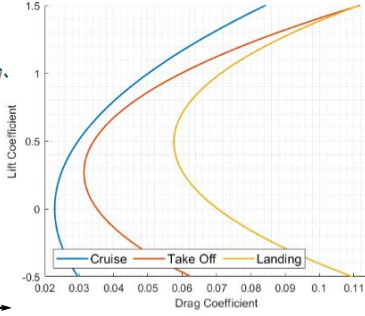
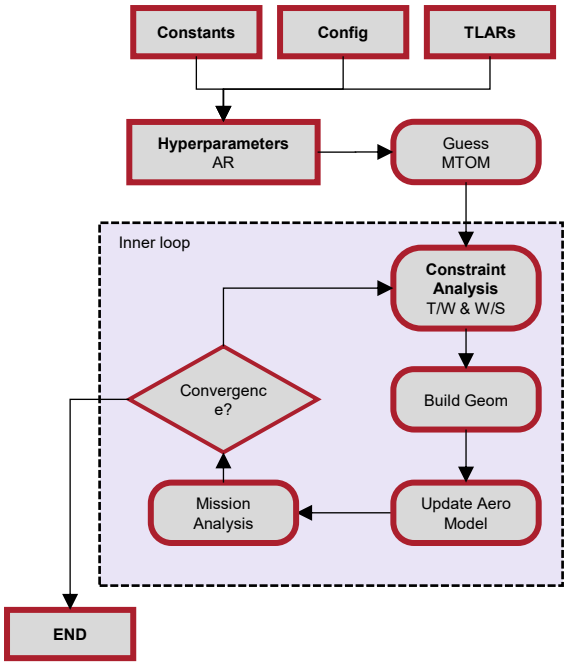


Figure 4: Design mission segment and fuel breakdown



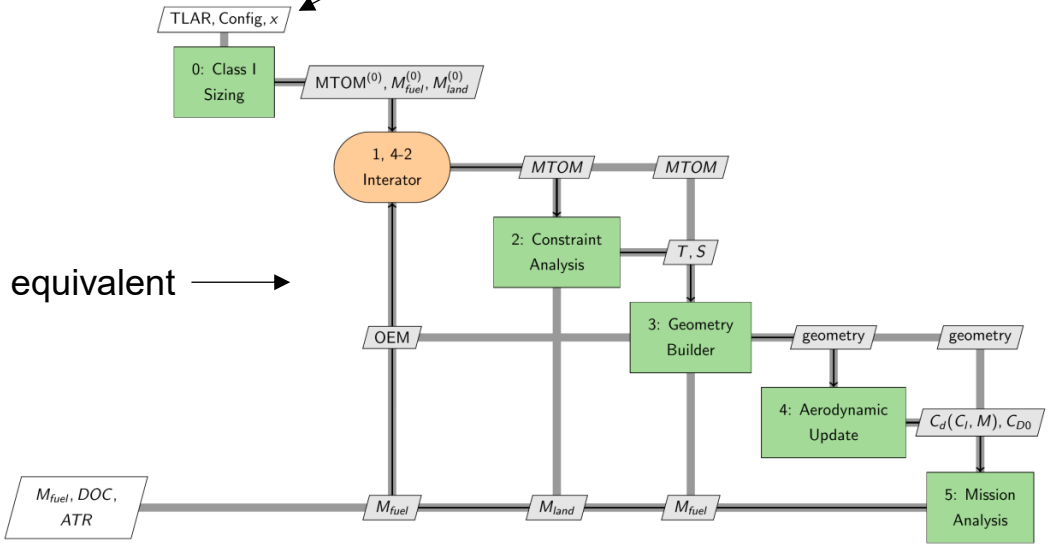


Interdependence (XDSM)



equivalent

‘x’ is a vector of our hyperparameters



University of BRISTOL

The Example Code

bristol.ac.uk

Some Notes

- These simple sizing tools are built on a lot of assumptions and '***magic numbers.***'
- You **must** calibrate against known aircraft
  - **This is the only way you can build confidence in your results**
  - For the same payload, design mission & hyperparameters as the B777F, you should get:
    - Similar OEM
    - Similar Trip Fuel Mass
  - Equally, how well do your tools scale?
    - If you change the payload/design mission, can you roughly match
      - A220/E195, A320/B737, A350/B777, A380/B747, etc…
    - It won't be perfect – but your trends should be reasonable
      - Otherwise, how will you assess fleet size?
- This may be tricky for your second concept – you'll have to factor in this uncertainty in your decision-making

Some Notes

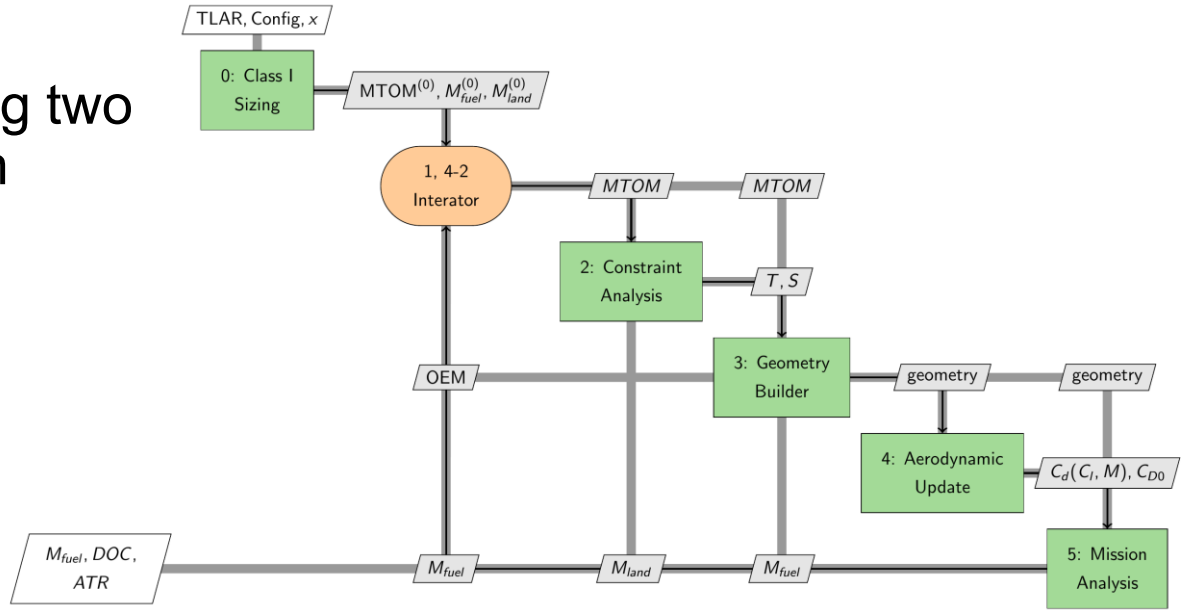
- **You are not designing an aircraft**
- **You are building a mathematical model of an aircraft.**
- The model is:
  - Calibrated to reproduce known performance
  - Used to study "*deltas*"
    - how performance, cost, and climate impact change when you vary assumptions and hyperparameters

- **What not to do**:
  - ❌Blindly use the tool without calibration
  - ❌Chase absolute realism
  - ❌Add complexity to fix a single discrepancy
- **What I'm looking for**
  - ✅Models *approximately* match known aircraft
  - ✅Assumptions clearly stated
  - ✅Hyperparameters identified with ranges justified
  - ✅Trends are physically sensible
  - ✅Conclusions are framed in terms of relative change, not absolute truth

"*Relative to a B777-like aircraft (sized with your tool), our concept achieves a x% larger … etc…*"

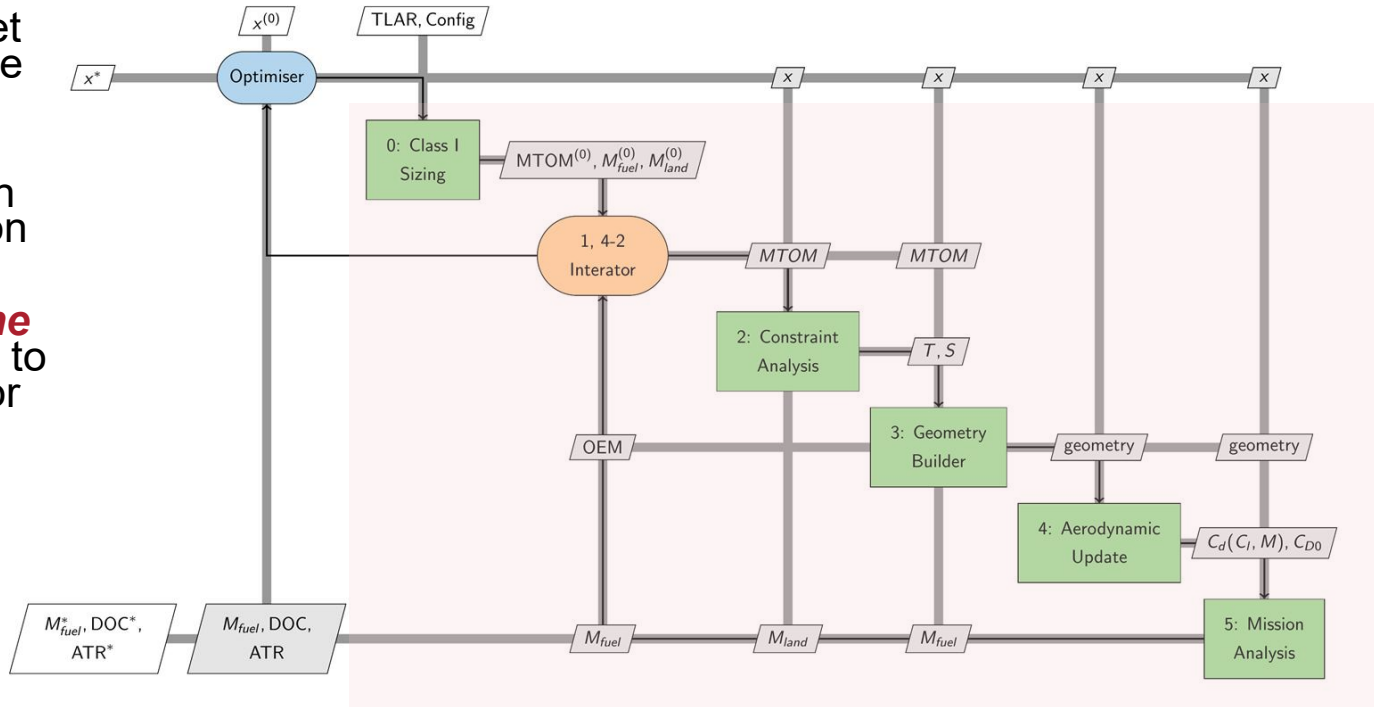
Sizing Versus Optimisation

- **Sizing**: For a *fixed set of hyperparameters*, build a valid aircraft
- As teams you are building two sizing tools (one for each concept)



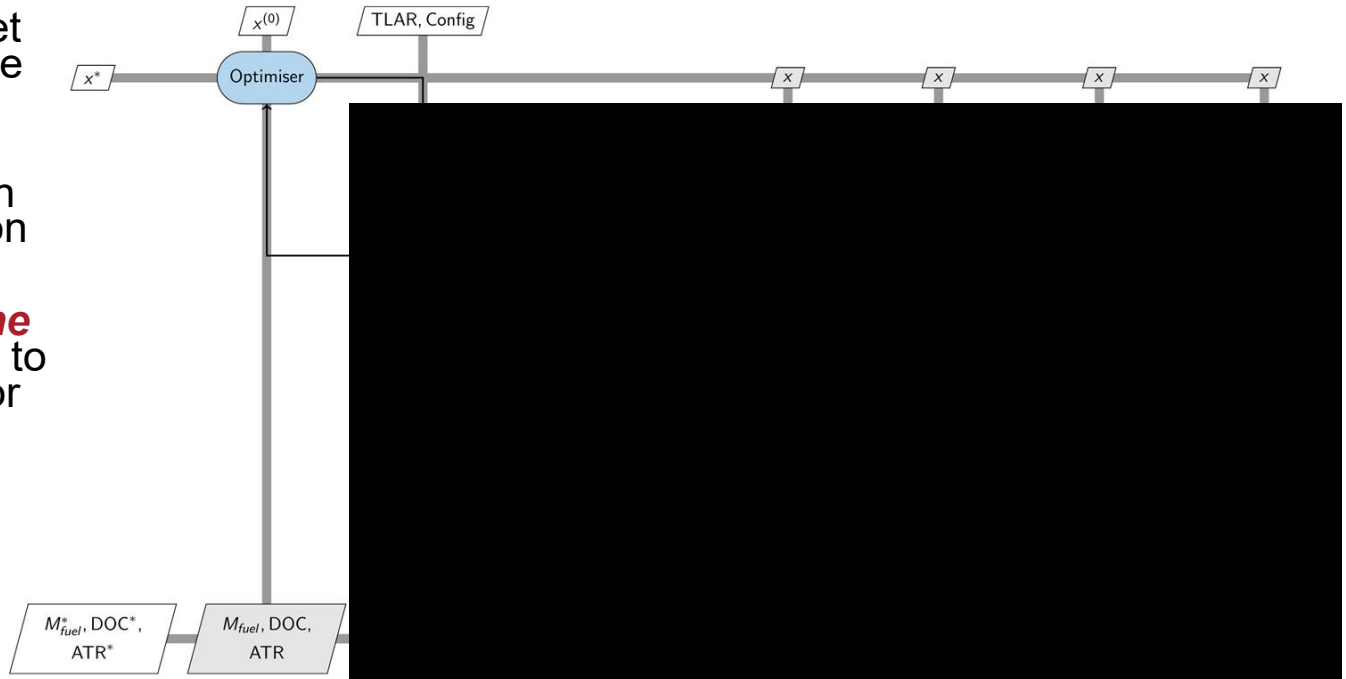
Sizing Versus Optimisation

- **Optimisation**: find the optimal value of hyperparameters to meet some objective (minimise DOC or ATR, for example…)
- The sizing process is run repeatedly in optimisation
- The optimisation engineering ***explores the design space*** and aims to find an optimal design for each sizing tool



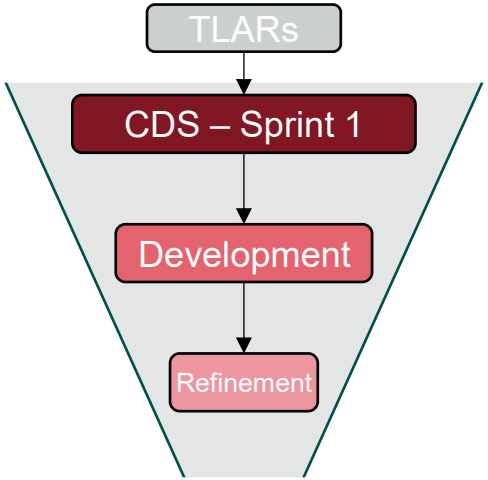
Sizing Versus Optimisation

- **Optimisation**: find the optimal value of hyperparameters to meet some objective (minimise DOC or ATR, for example…)
- The sizing process is run repeatedly in optimisation
- The optimisation engineering ***explores the design space*** and aims to find an optimal design for each sizing tool



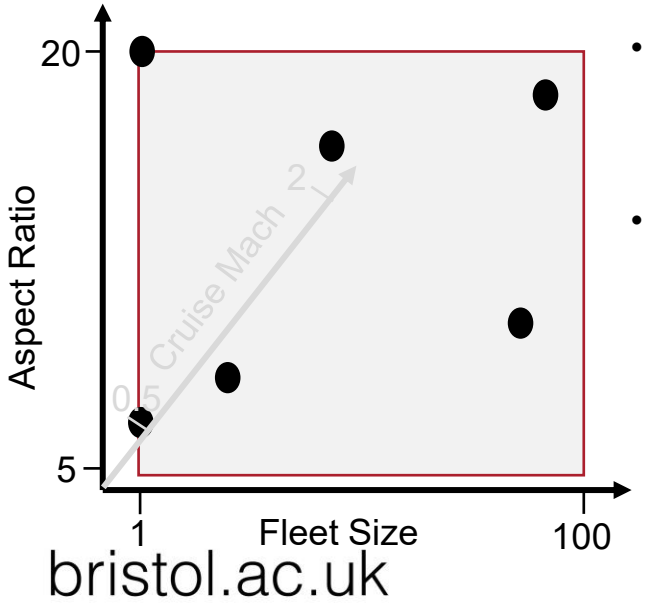
Classification Confusion

- *"How can I know if I've completed class I sizing, and when should we move onto class II sizing?"*
- Don't worry too much about the "*classification*" of your sizing tool!
- What I want to see is an **appropriate progression in the fidelity** of your sizing tools
  - e.g., from the development to refinement sprint
  - But what do I mean by this?



Exploring the Design Space

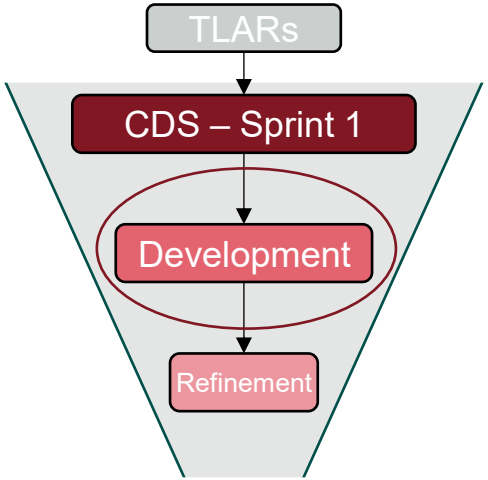
- In the development phase, you need to quickly build two ‘sizing tools’
  - You need to *quickly* develop each discipline to be robust across the entire **design space**



- Making sure each discipline works well across all combinations of parameters will be difficult!
- This is why you need lower fidelity tools in the development phase
  - Adding panel buckling, complex aerodynamic models, or sizing control surface is ~~difficult~~ at this scale

Counter-productive

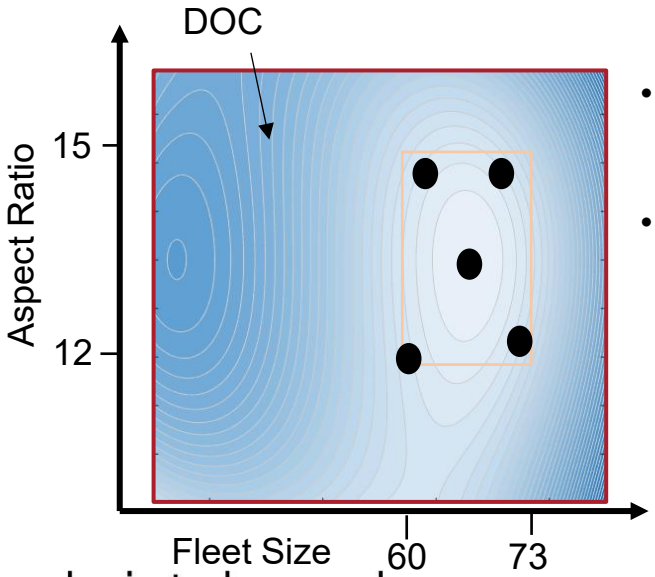
Start with lower-fidelity tools so that we can cover the entire design space



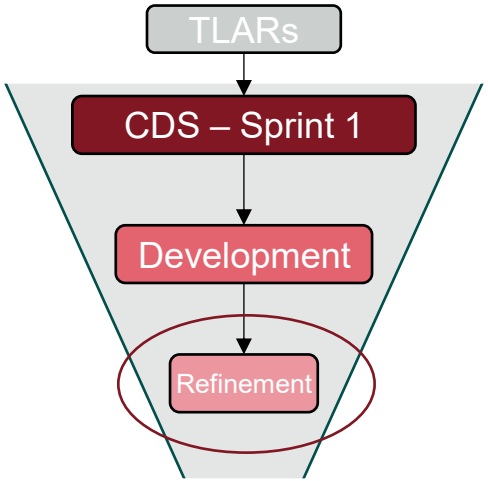
Refining the Design Space

- In the refinement sprint, you can:
  - Look at a single concept
  - Narrow the design space around the current optimum
    - Remember, the optimum may still move!

End with higher-fidelity methods to refine the sizing tool around a likely optimum!



- During refinement, there is more certainty in the size and shape of the aircraft
- This makes it easier to conduct higher fidelity analysis, e.g.
  - Develop a high lift system
  - Size control surfaces
  - Size wingbox structure
  - Assess where fuel will go
  - Etc…



University of BRISTOL

Questions?

bristol.ac.uk

Code Management

bristol.ac.uk

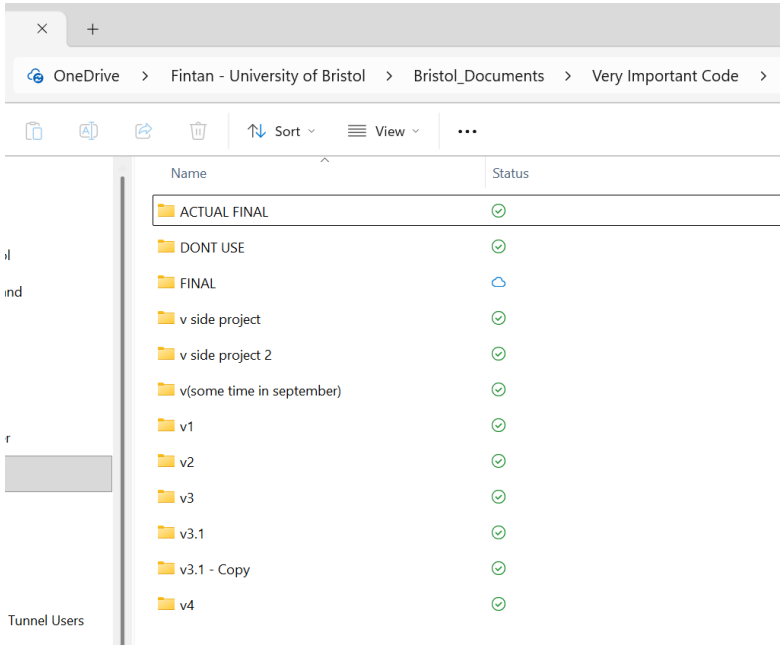
Why Version Control (Source Control)?

- **The Perils of Teamwork:**
  - Multiple people editing the same files can overwrite each other's work.
  - Hard to track who changed what, when, and why.
  - Integrating code at the end leads to conflicts and lost work.
  - Without history, debugging becomes guesswork.
  - Collaboration slows when everyone is afraid to touch shared files.

Why Version Control (Source Control)?

Lone work

- **The Perils of ~~Teamwork:~~**
  - Local files get overwritten, deleted, or corrupted with no recovery.
  - Impossible to track progress or compare versions over time.
  - Hard to experiment safely without breaking working code.
  - Copy-paste backups (v1, v2, final_FINAL.zip) quickly become unmanageable.
  - No safety net → no confidence.



There Must Be An Alternative?!

- Five Key Properties of a Good Version Control System:
  - **Versioning**
    - Every change is recorded; you can go back in time (Nothing is truly lost).
  - **Isolation**
    - Work on new features safely without breaking the main code.
  - **Mergeability**
    - Bring everyone's changes together without chaos.
  - **Traceability**
    - Know who changed what, when, and why.
  - **Memory**
    - Stores many versions efficiently

Option 1: God Mode

Versioning	
Isolation	
Mergeability	
Traceability	
Memory	

- Everyone codes locally → one person integrates everything!
  - **Simple**, no software needed, easy for 6 of you!
  - One person (**Integrator-in-Chief**) resolves all differences.
  - Works if the team is small and the **code is loosely coupled**.
  - But… creates bottlenecks, delays, and huge pressure on the integrator.
  - Merge conflicts become IiCs problem...

Option 2: Shared Drive + Daily Freeze Frames

- **Shared folder** with strict versioning rules
  - Everyone edits files in a common directory.
  - Each day, the PM (or IE) creates a **“freeze frame” snapshot** (e.g., 2025-02-07_Code_Freeze).
  - Gives a **minimal form of history** and rollback.
  - Easier than industry standard (Git…), **better than chaos**.
  - Still prone to overwrites, missing changes, and silent conflicts.

Option ~~3~~: Not Options…

- Emailing code files.
- Sending code snippets or screenshots in chat.
- Overwriting files without warning.
- Having multiple people editing the same file simultaneously.
- Using "*final_final_v7_reallyFINAL.zip*" as version control.

Versioning	
Isolation	
Mergeability	
Traceability	
Memory	

Enter Git...

Versioning	
Isolation	
Mergeability	
Traceability	
Memory	
Simplicity	

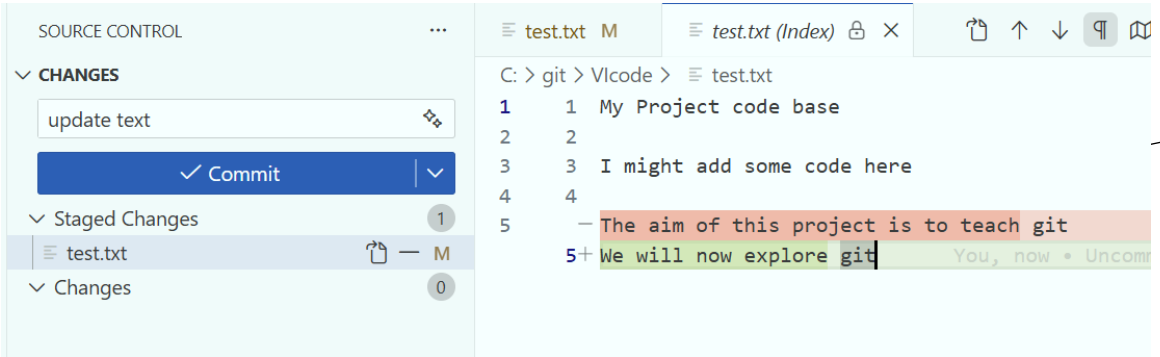
- Professional solution with a *steep* learning curve
  - You don't have to use it! But....
- **Git**
  - Tracks every change with full history and accountability.
  - Allows safe branching and merging for parallel work.
  - Prevents accidental overwrites and lost work.
  - Works for individuals and teams without bottlenecks.
  - Standard tool used in engineering, research, and industry — the safest choice.

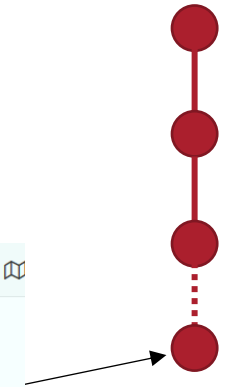
Git Basics

- Git is not a cloud folder; it's a "*history engine.*"
- Instead of saving files, Git saves **a history of file changes**

Git Basics

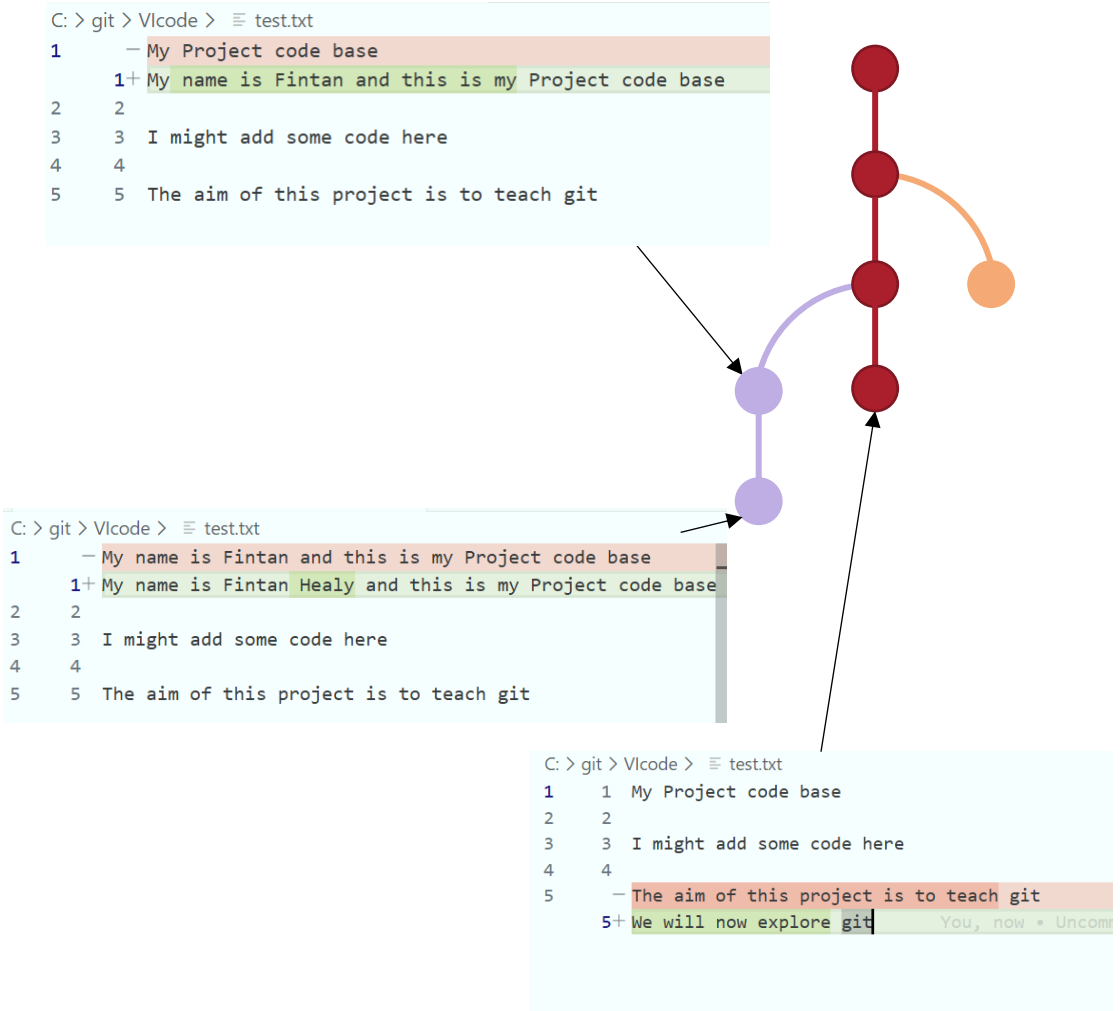
- When you edit files, you can
  - **stage** changes
  - then **commit** changes to the history





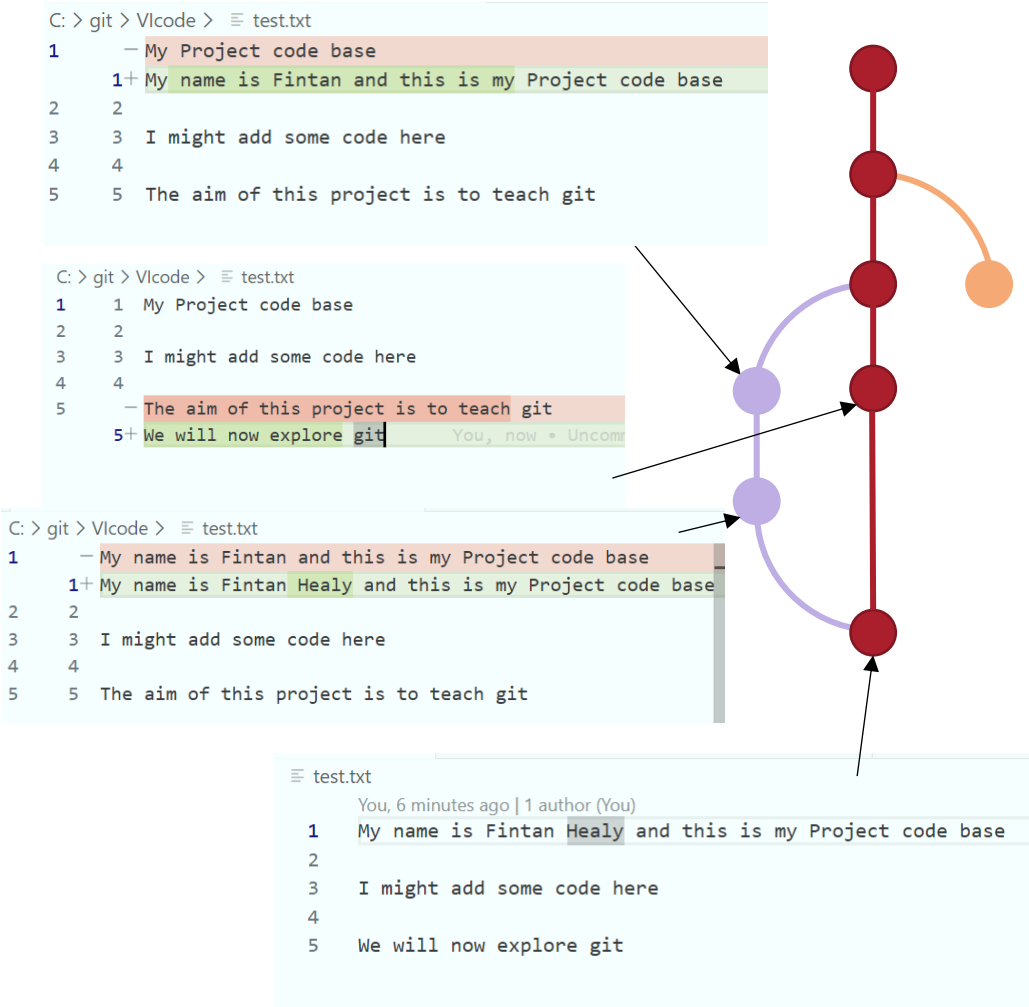
Git Basics

- You can **branch** code at any point in the history
  - You can then commit onto this branch
- Branches are cheap
  - Don't name after yourself, name after a feature!
    - Not "*FintansBranch*"
    - "*ConstraintAnalysis*"
  - You could even name them with backlog IDs?
- Branches are not copies, they are **parallel timelines**



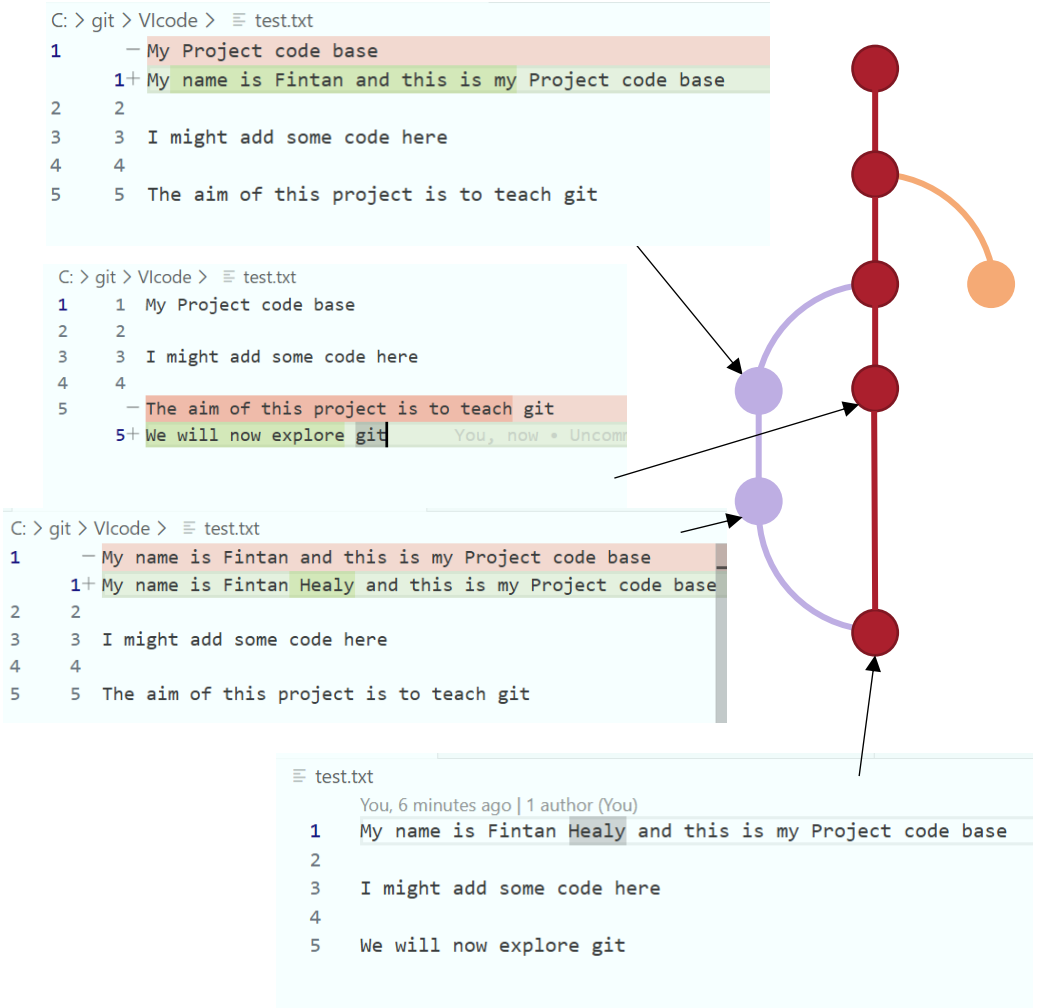
Git Basics

- You can **merge** branches to combine change history
  - Sometimes you may have to deal with merge conflicts…
  - Merging keeps the main branch updated with completed work
  - But you can also merge to ‘side’ branches!



Git Basics

- This is all **local**
- But you can **push** your file history to a server (Github)
- Others can then **pull** your changes to there local repositories
- **Be careful:**
  - Have you only committed work locally?
  - You need to push to the server for others to see it!



Some Notes

- Git is for text files!
  - Don’t put binary files on unless you have to!
- You can restrict what types of files get committed
  - See “.gitignore” files
  - This stops you from adding binary files (data, or pictures) to the repo
- On the server, you can “**Protect**” certain branches to stop people accidentally pushing to your main branch
- You **don’t have** to use git:
  - Perhaps only one uses git (in god mode)
  - Or maybe you use it all locally, but not connected
  - It’s a good skill to learn though!

Recommended Steps to Get Started

- Install git locally
- Create a GitHub account
- CADEM0016 repositories
  - Email me your GitHub username, and I'll add you to the organisation
- Clone your groups Repository locally
  - Create a new branch
  - Make changes
  - Commit (locally)
  - Push (move (push) commits to organisation)
  - Pull (get (pull) changes from organisation)
  - Merge
  - Pull Request to master…

- Tips
  - Branches are cheap
  - Commit little and often
  - Unit test code before merging with master!

- Plenty of online guides, but I'll show you some basics now!
- GitKraken is a great tool for git management!

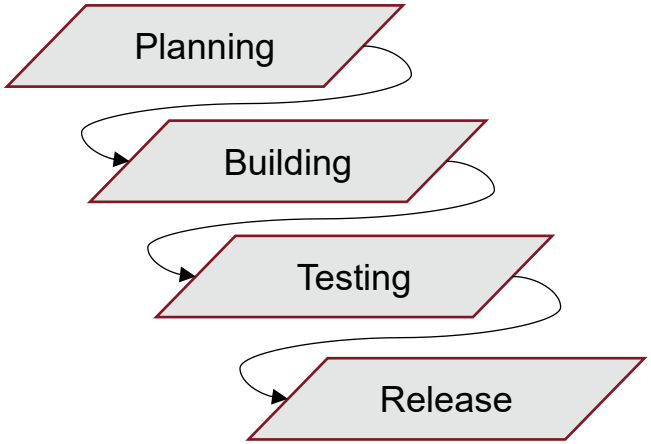
University of BRISTOL

Project Management & Group Work

bristol.ac.uk

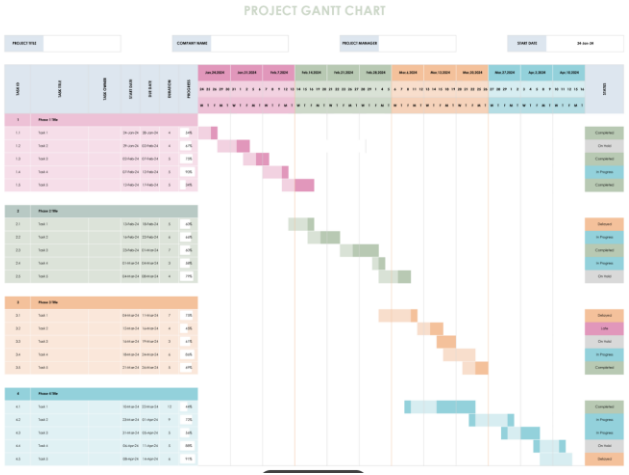
Traditional Workflows

Traditional Waterfall
3-6 months?

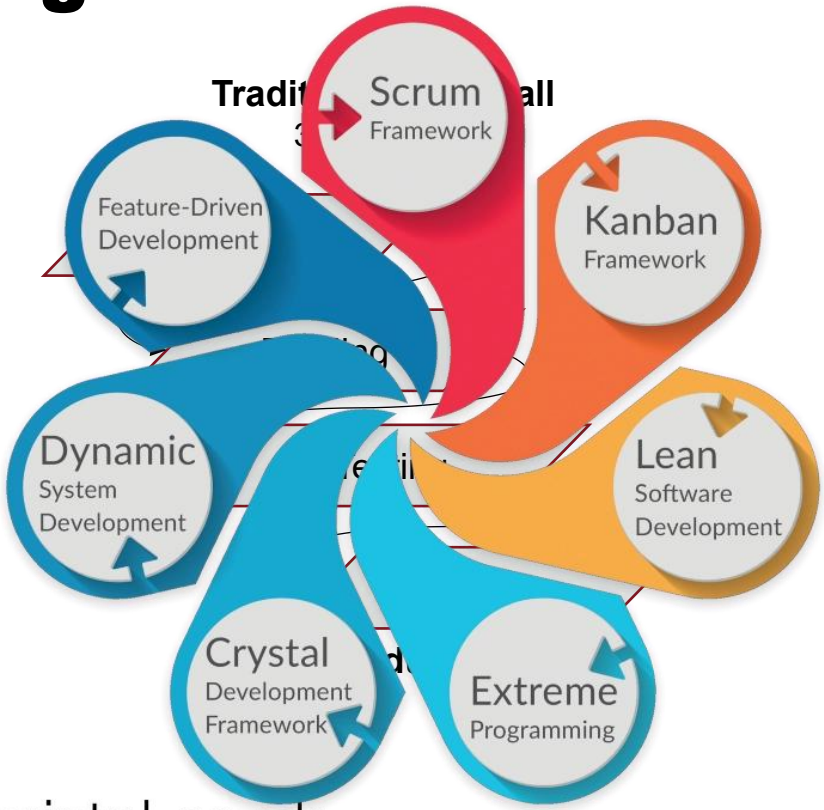


Product Release

- **Traditional Waterfall** Project Management
  - Linear and Sequential
  - Upfront planning
  - Gated Progression
  - Documentation Driven

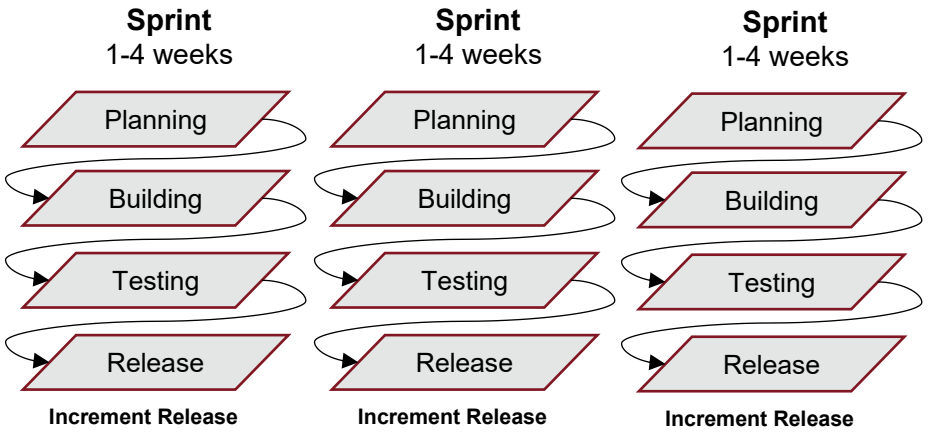


Agile Workflows



bristol.ac.uk

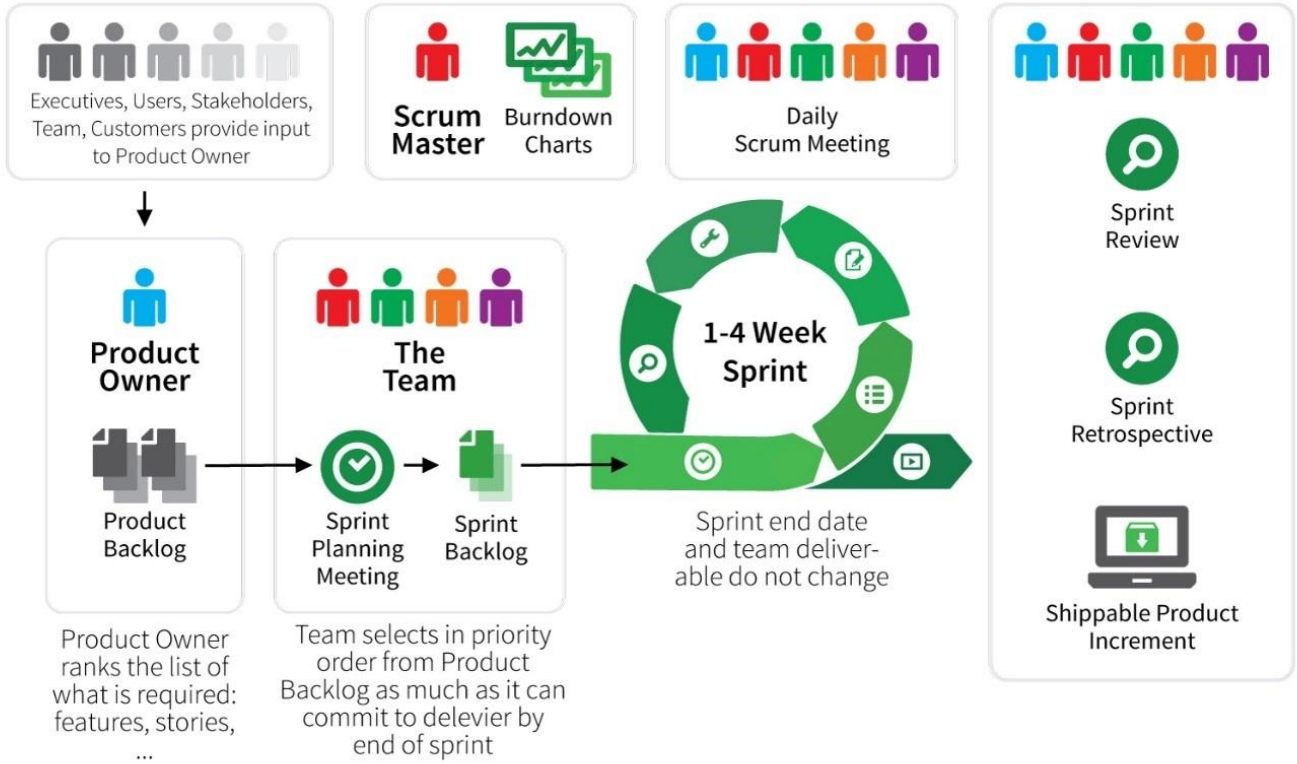
Introducing Scrum



- Scrum is an agile framework that
  - **Splits work into small, focused increments** to deliver usable progress early and often.
  - **Adapts continuously** through regular inspection, feedback, and reprioritisation.
  - **Empowers self-organising teams** to collaborate openly and take collective ownership of outcomes.

https://www.scrum.org/

Scrum Framework at a Glance



3x Roles

Product Owner

Scum Master

Dev. Team

3x Artifacts

Product Backlog

Sprint Backlog

Burndown Charts

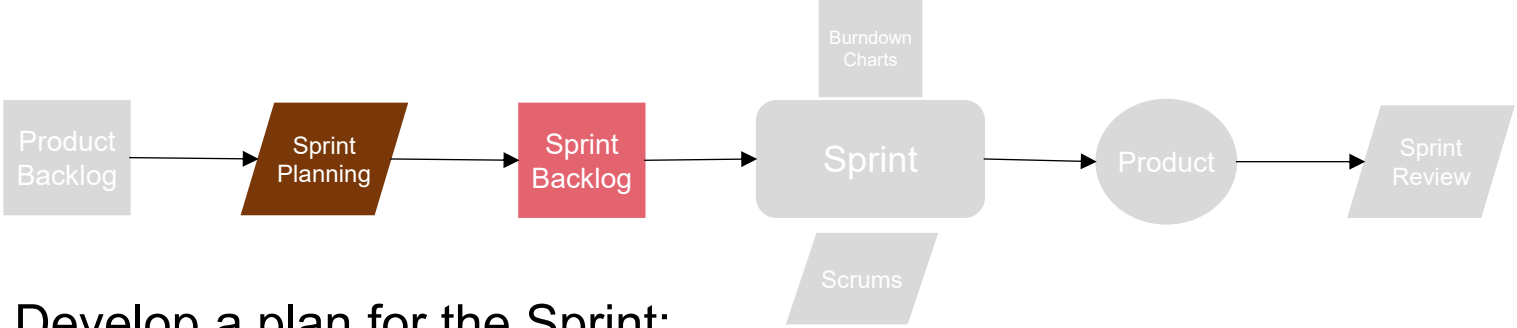
3x Ceremonies

Sprint Planning

Daily Scrum

Sprint Review

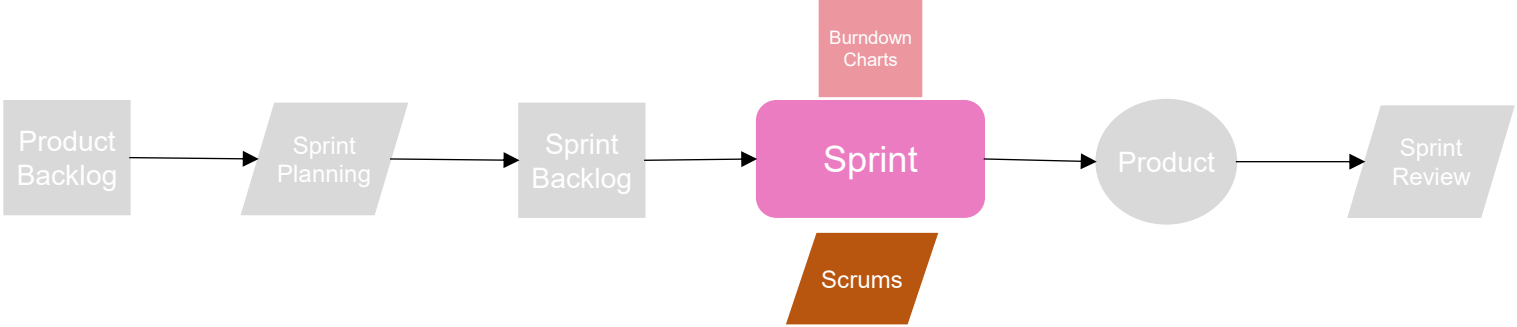
https://www.scrum.org/



- Develop a plan for the Sprint;
  - Define Goals
  - Select backlog items to complete in the sprint
    - Break work into small, meaningful tasks
    - Estimate effort and capacity
    - Identify Dependencies and risks early
    - Agree on “Definition of Done.”
  - PM Then Maintains the Sprint Backlog

- *What makes a good backlog item*?
  - Split large tasks (“Epics or Features…”) into small, meaningful tasks.
  - Each task should involve 0.5-2 days of effort
  - Each team member should have 6-10 backlog items per sprint
    - But don’t see that as a target!

https://www.scrum.org/



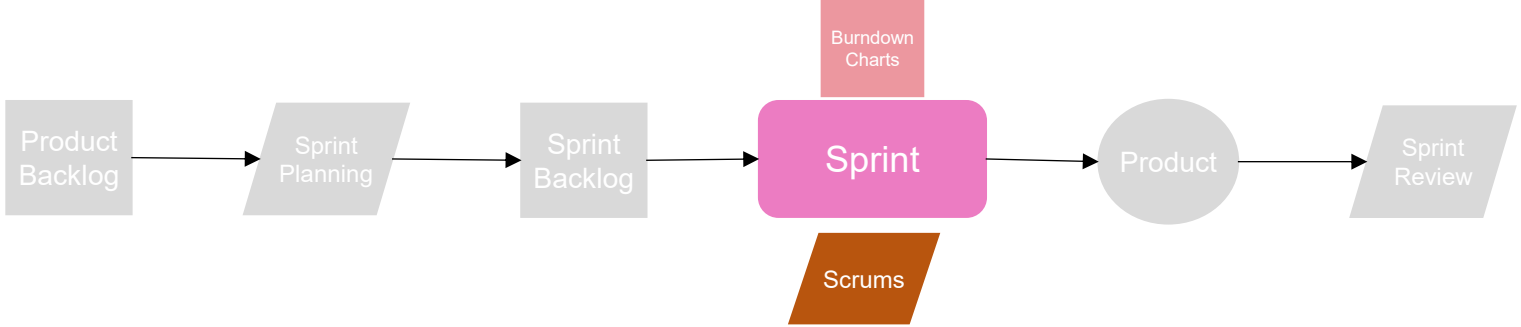
- **Sprint** – Fixed-length working period
- **Scrums –** regular meetings where team members take turns to:
  - share progress
  - Identify blockers
  - Adjust task ownership
  - Identify new tasks for backlog

Also known as "stand-up meetings."

- Aim for 2/3 scrums a week

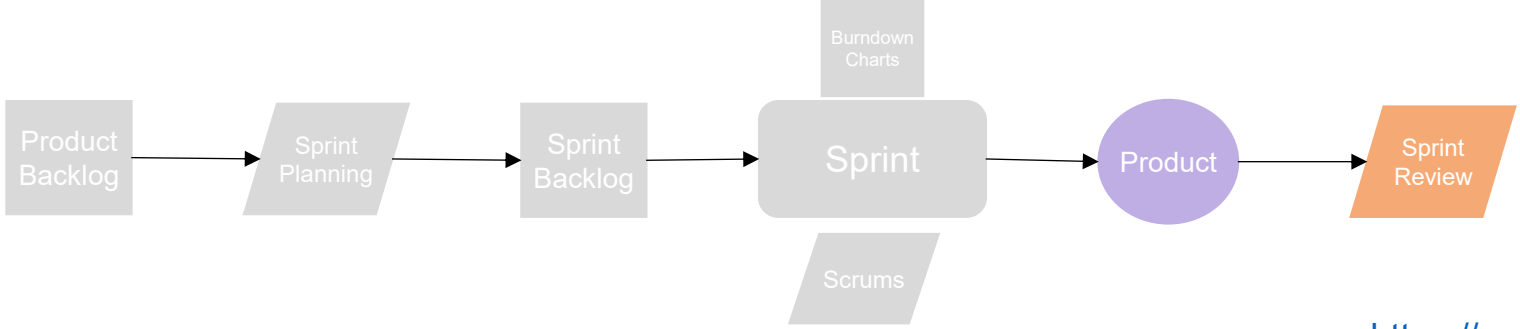
- **Scrum Master Responsibilities**
  - Facilitate team coordination
    - Communication identifies blockers
  - Remove impediments
    - Act quickly if things are slowing progress
  - Protect against scope creep
    - Protect against new tasks
  - Monitor Progress
  - Support healthy Agile practice
    - Split large tasks
  - Promote collaboration
  - Prepare for review

https://www.scrum.org/



- **Burndown charts**
  - Help you track the rate at which Backlog items are being cleared
    - Y axis could be items or hours …
  - Can help identify if you're stalling
    - Are backlog items too big, or is there a genuine issue?
  - Reinforces team accountability
  - Provides evidence for how the sprint unfolded
  - Helps improve future sprints

https://www.scrum.org/



https://www.scrum.org/

- Review how the sprint went as a team:
  - Discuss any scope adjustments and unexpected findings/dynamics from the sprint
  - Update product backlog
  - Used to help the next sprint run more smoothly!

- **PM Expectations:**
  - Read how to be an effective Scrum Master
  - Manage Product / Sprint Backlog
  - track task completion
  - Organise 2/3 scrums a week
- **CDR**
  - Burndown chart – discuss sprint progress, key blockers, improvements for next sprint
  - FHA* for Sprint 3
- **FEDR**
  - Burndown charts for sprints 2 & 3
  - Used as talking points to discuss team dynamics, identification of issues, remedies, etc.
  - FHA* for a future design team

University of BRISTOL

Risk Management

bristol.ac.uk

Functional Hazard Analysis (FHA)

- First step for System / Project Safety
- Standardised process (MIL-STD-882)
- Typically used to assess aircraft-level failures (hazards)
- You will use to assess Design level Failures
- Each identified failure is scored on:
  - Severity
  - Probability
- Help you identify key risks and mitigate probability

SEVERITY CATEGORIES		
Description	**Severity Category**	**Mishap Result Criteria**
Catastrophic	1	Could result in one or more of the following: death, permanent total disability, irreversible significant environmental impact, or monetary loss equal to or exceeding $10M.
Critical	2	Could result in one or more of the following: permanent partial disability,injuries or occupational illness that may result in hospitalization of at least three personnel, reversible significant environmental impact, or monetary loss equal to or exceeding $1M but less than $10M.
Marginal	3	Could result in one or more of the following: injury or occupational illness resulting in one or more lost work day(s), reversible moderate environmental impact, or monetary loss equal to or exceeding $100K but less than $1M.
Negligible	4	Could result in one or more of the following: injury or occupational illness not resulting in a lost work day, minimal environmental impact, or monetary loss less than $100K.

PROBABILITY LEVELS			
Description	**Level**	**Specific Individual Item**	**Fleet or Inventory**
Frequent	A	Likely to occur often in the life of an item.	Continuously experienced.
Probable	B	Will occur several times in the life of an item.	Will occur frequently.
Occasional	C	Likely to occur sometime in the life of an item.	Will occur several times.
Remote	D	Unlikely, but possible to occur in the life of an item.	Unlikely, but can reasonably be expected to occur.
Improbable	E	So unlikely, it can be assumed occurrence may not be experienced in the life of an item.	Unlikely to occur, but possible.
Eliminated	F	Incapable of occurence. This level is used when potential hazards are identified and later eliminated.	Incapable of occurence. This level is used when potential hazards are identified and later eliminated.

FHA Example

- **Job of the PM -** maintain list of risks, work with the team to reduce severity or probability
- **Job of Team -** help PM identify risks and mitigate them
  - Risks can appear at any point in the project!
- The point of this process is to help you identify and manage risks

RISK ASSESSMENT MATRIX				
SEVERITY / PROBABILITY	Catastrophic (1)	Critical (2)	Marginal (3)	Negligible (4)
Frequent (A)	High	High	Serious	Medium
Probable (B)	High	High	Serious	Medium
Occasional (C)	High	Serious	Medium	Low
Remote (D)	Serious	Medium	Medium	Low
Improbable (E)	Medium	Medium	Medium	Low
Eliminated (F)	Eliminated			

Function	Hazard	Effect	Severity	Probability	Risk	Mitigation
Code integration	Merge conflicts deletes key functions	MDO tool breaks	2	C	S	Ensure members commit work often
Code integration	Merge breaks the current functionality	MDO tool breaks	2	B	H	Members push commits to a develop branch – code must pass unit tests to get to main
Wing Sizing	Mass over estimated at large ARs	High Ars designs unfairly penalised	3	C	M	Cross correlate mass increase with AR to research

Example Risks

- Examples of other risks:
  - Wing mass underestimated due to lack of gust analysis
  - OEI cross-wind requirements not sized, Tailplane too small
  - Fuel price fluctuations may affect optimal design
  - Mission analysis is unable to handle step climb scheduling
  - Integration risk in version control (tool becomes unmaintainable)

How The PM will Use FHA

- **CDR** – Identify risks for Sprint 3
- **FEDR** – Identify risk for a future engineering team extending your design or refining your MDO tool.
- **Poster** – Talk about how risks evolved over the sprint cycles

University of BRISTOL

Conclusions

bristol.ac.uk

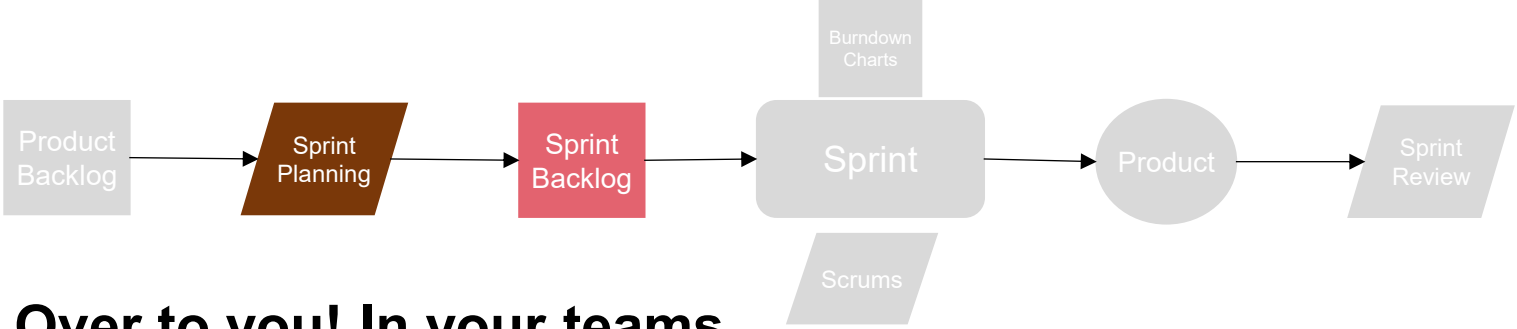
Conclusions

- Reviewed MDO tool
- Talked about code management strategies
- Reviewed the Scrum framework
- Reviewed Functional Hazard Analysis

University of BRISTOL

Questions?

bristol.ac.uk



- **Over to you! In your teams,**
  - Have a go at building your backlog.
  - Discuss tasks and break them down into manageable chunks
  - Organise your first scrum
- **PMs** – Think how you want to organise / track backlog items

- *What makes a good backlog item*?
  - Split large tasks (“Epics or Features…”) into small, meaningful tasks.
  - Each task should involve 0.5-2 days of effort
  - Each team member should have 6-10 backlog items per sprint
    - But don’t see that as a target!

https://www.scrum.org/